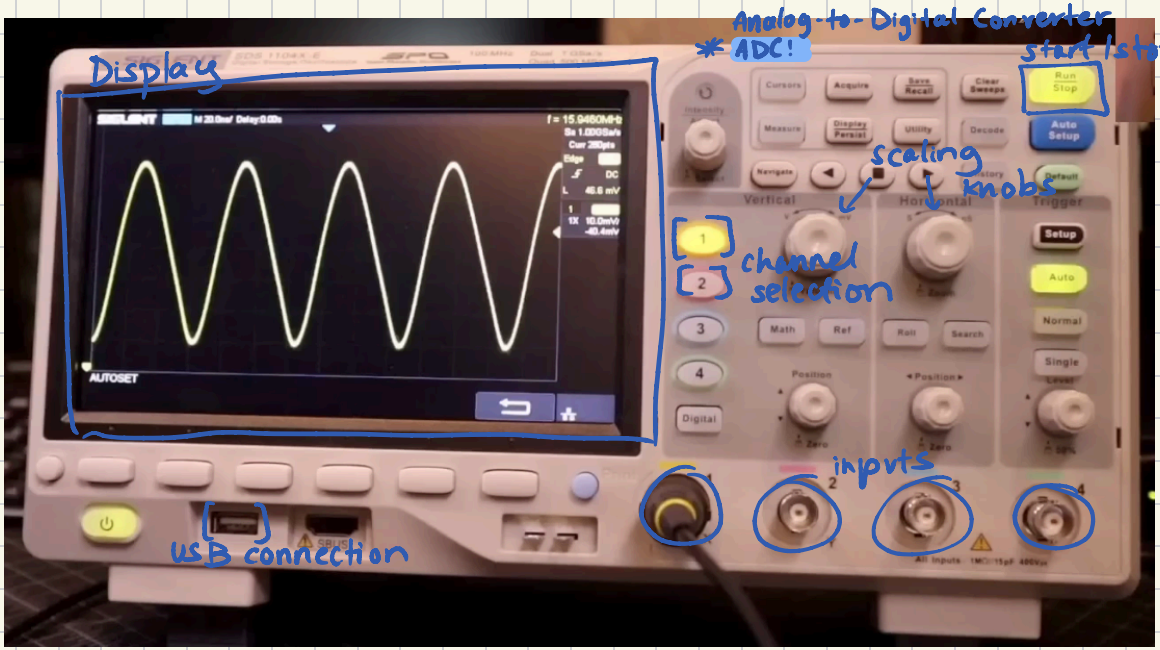
Silly scope

Documentation Notebook

By Aaruni Singh



Step 1: Take abstract idea of an oscilloscope and mapping it out into high level substructures/components

Scope:

- → 2 channels
- → Graphic Display shows 1 channel
- → 0V to 3.3V range

Hardware requirement
Firmware requirement

↳ (simpler scope atm, future iteration could be adding analog stage with op-amps for offset or to scale)

- → Accurate to 500 KHz

↳ ADC that is used need sample rate of this

- → Data Streaming / PC logging

USB connection
data streaming/logging protocol

↳ record and log + potentially add other channel if streaming in real time)

- → Buttons for Channel Activation
- → Button for Start/Stop
- → LED indicators for power, channel activity, recording, etc.)

STM32L432KC → STM32 Cube Mx
→ 12 bit ADC 5Msps (10x scope) → built in
→ USB support
→ doesn't have many pins
↳ ∅ use 8-bit parallel display → use 4 wire SPI
DISPLAY: 192x96 panel
↳ controller
↳ code samples to help shape communication driver
↳ code give illustration of timing diagram so we can write our own communications library
↳ draw own schematic symbol

Custom PCB Design Process

1. BOM
↳ microcontroller, lcd, other peripherals based on scope
2. Power * Electrical Specifications SCHEMATIC
↳ power rails ⇒ only need one line: 3.3 V

*Critical mistake: forgot decoupling capacitors

3. Decoupling Capacitor SCHEMATIC
↳ on/off creates noise on input voltage rail
↳ add local power supply to help stabilize it
↳ AN4555 ⇒ Recommendations ⇒ decoupling
↳ btwn VDD and Vss → 100nF ceramic on power rails
VDD → Vss (GND)
↳ for LCD → ∅ prt decoupling cap bc already on PCB (assumption)

4. STM32CubeMx ⇒ pin definition on microcontrollers
↳ SPI → Transmit only Master (∅ getting back from LCD)
↳ Hardware NSS Signal → Disable (use software chip select instead)
↳ GPIO output: Chip Select
↳ ADC → single-ended differential (reference to GND)
↳ Buttons → Interrupts (need dedicated number for each)
↳ LEDs → GPIO Outputs

↳ USB enable ⇒ Communication Device class
↳ Virtual Port Com
↳ give serial port that we can open and read data thats going to be streamed to us

↳ Debug Interface → to program software onto it
→ AN455 → gives use note on debug interfaces

↑ GPIO Pins ⇐ JTAG vs SWD ⇒ requires only 2 GPIO wires
(small microcontroller → GPIO constrained)

1. NRST ; 2. SWCLK ; 3. SWDIO

↳ Debug UART → Asynchronous

5. Update SCHEMATIC
↳ Add new pin configurations as per STM32CubeMX
↳ Add LEDs (connected directly onto microcontroller)
↳ ∅ ∞ current from GPIOs → add resistors
↳ avoid burnout by limiting current
$\sum I_O$ (pin): total output current sourced by sum of all IIOs
≤ 100 mA

$$R = \frac{V_{battery} - V_{LED}}{I_{LED}} \Rightarrow I = \frac{\Delta V}{R}$$

(datasheet) ↗ initial assumption

$$\frac{3.3\,V - 2.1\,V}{200\,\Omega} = 6\,mA \Rightarrow \boxed{18\,mA\ total}$$

↳ Boot pulled to GND

6. Power Input Delivery Pipeline SCHEMATIC
↳ USB provides 5 V ⇒ start there
↳ add voltage regulator to scale down to 3.3 V
↳ selected: current limit output of 1 A
(assumption: should be enough)

↳ Voltage regulator ⇒ add decoupling capacitors (datasheet)
↳ EN pulled up to 5 V: USB connect ⇒ 3.3V output
↳ pull-up resistor to limit current

↳ need USB 2.0 connector
↳ V_{BUS} = 5V ; DP|DN ⇒ microcontroller
↳ pull down on CC_1, CC_2 w/ 5.1k resistors
↳ terminate CC pins w/ pull down resistors
(upstream facing port : peripheral)

7. Wire Up LCD SCHEMATIC
↳ interfacing guide on BuyDisplay ⇒ use that

SI, SO, CS, SCLK ⇒ ø use bc font display (ø need)
↳ need graphic display ⇒ wired for such purpose

↳ ø in datasheet but added pull-up resistors (assumption)
↳ on schematic ⇒ share resistor bc pins are unused

↳ add 2 new signals (LCD_A0, LCD_RST)
↳ allocate on cubeMX

8. Add Push Buttons SCHEMATIC
↳ GNDed and pull-up resistor to 3.3V
↳ signal is high unless button is pushed
↳ then pulled to low

9. Signal Input Stage SCHEMATIC
↳ BNC connectors ⇒ internal going to ADC (2 channels)

10. Debug Interface SCHEMATIC
↳ SWD: Connector + Programmer that supports STM32
↳ ST-LINK-V3 MINIE ⇒ 14 pin connector ⇒ 14 pin header
↳ VTREF ⇒ VDD (3.3V) ; #RST ⇒ NRST (STM)
↳ VCP RX/TX ⇒ Debug UART ↳ pulled high bc program will pull low